

Artificial Intelligence Agents

Core Concepts and Design Patterns for Developing Agentic AI

Alessandro Suglia



Outline

- Aim: provide an overview of AI agents from traditional formulations to more modern language-enabled AI agents
- Outline
 - Provide a definition of an AI agent including skills and capabilities
 - Review current language-enabled AI agents (aka LLM-based AI agents)
 - Compare and contrast these approaches
 - Review advanced prompting techniques
 - Study agentic design patterns for production-ready AI agentic systems

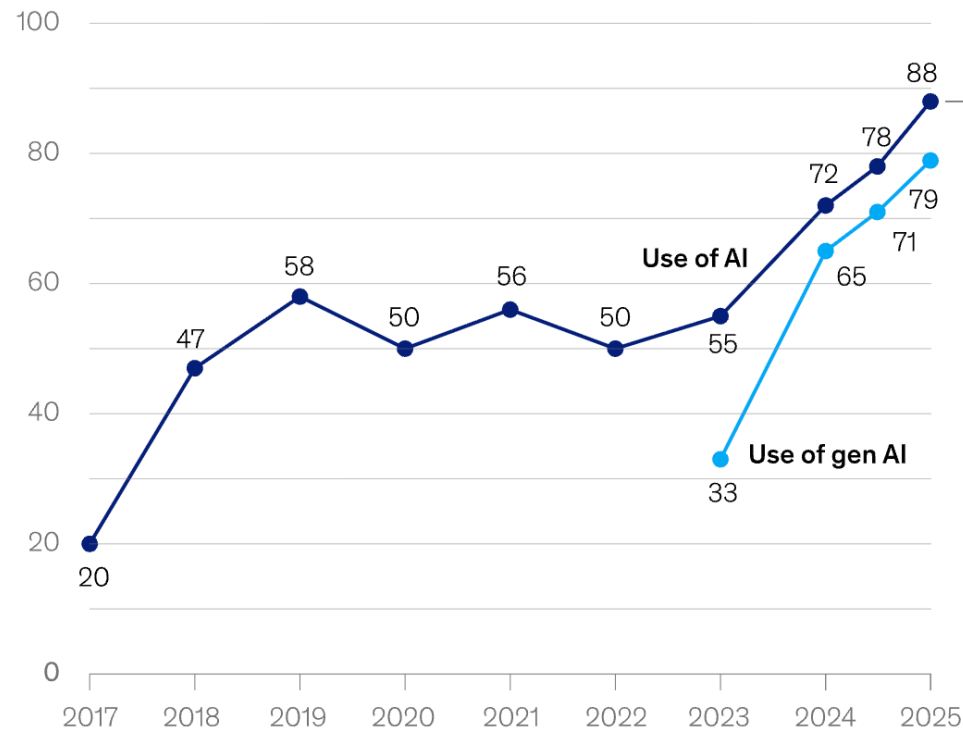
BACKGROUND ON AI AGENTS

Definitions, common knowledge and pitfalls

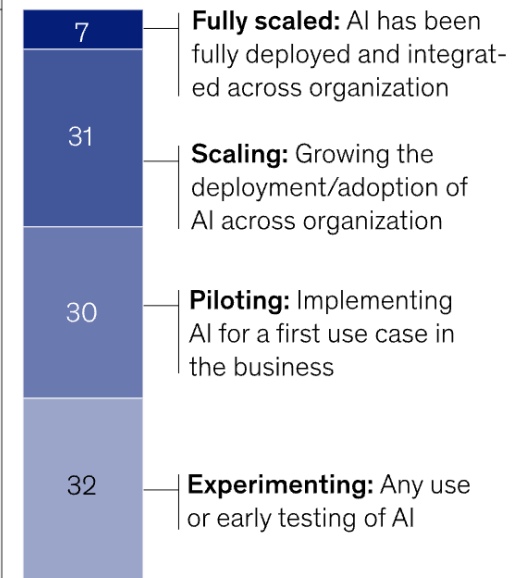
Reported use of AI in at least one business function continues to increase.

Use of AI by respondents' organizations, % of respondents

Organizations that use AI in at least 1 business function¹



Phase of AI use among organizations using AI in 2025



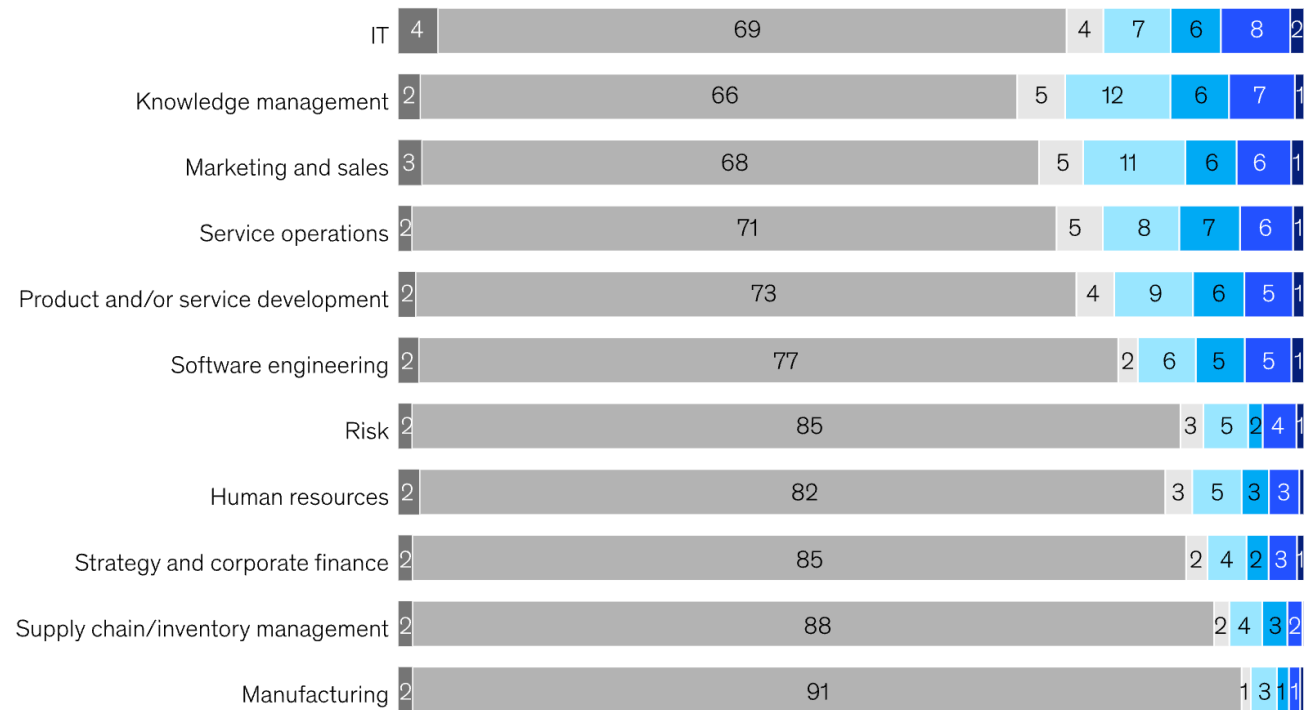
¹In 2017, the definition for AI use was using AI in a core part of the organization's business or at scale. In 2018–19, the definition was embedding at least 1 AI capability in business processes or products. From 2020, the definition was that the organization has adopted AI in at least 1 function, and in 2025, the definition was regular use of AI in at least 1 function.

Source: McKinsey Global Surveys on the state of AI, 2017–25

No more than 10 percent of respondents report scaling AI agents in any individual function.

Phase of AI agent use at respondents' organizations, by business function,¹ % of respondents (n = 1,933)

■ Don't know ■ Not at all ■ Planning to use within year ■ Experimenting ■ Piloting ■ Scaling ■ Fully scaled



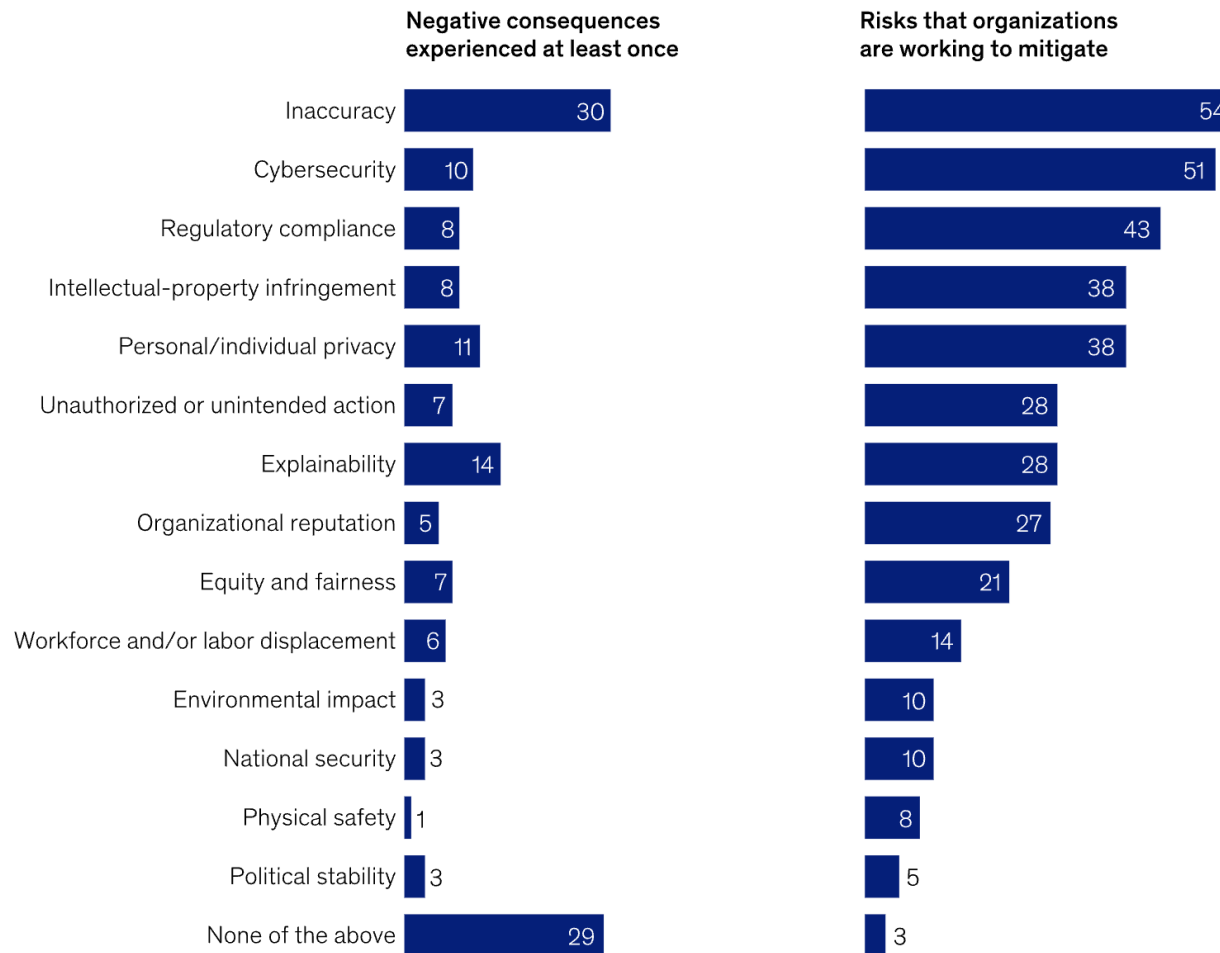
Note: Figures may not sum to 100%, because of rounding.

¹Question was asked only of respondents who reported regular use of AI in the respective functions and was rebased to reflect the total sample.

Source: McKinsey Global Survey on the state of AI, 1,993 participants at all levels of the organization, June 25–July 29, 2025

Inaccuracy is the AI-related risk that respondents most often say their organizations have experienced and are working to mitigate.

Negative consequences and risk mitigation in the past year,¹% of respondents (n = 1,753)



Inaccuracy is the AI-related risk that respondents most often say their organizations have experienced and are working to mitigate.

Negative consequences and risk mitigation in the past year,¹% of respondents (n = 1,753)



How do we define AI Agents?

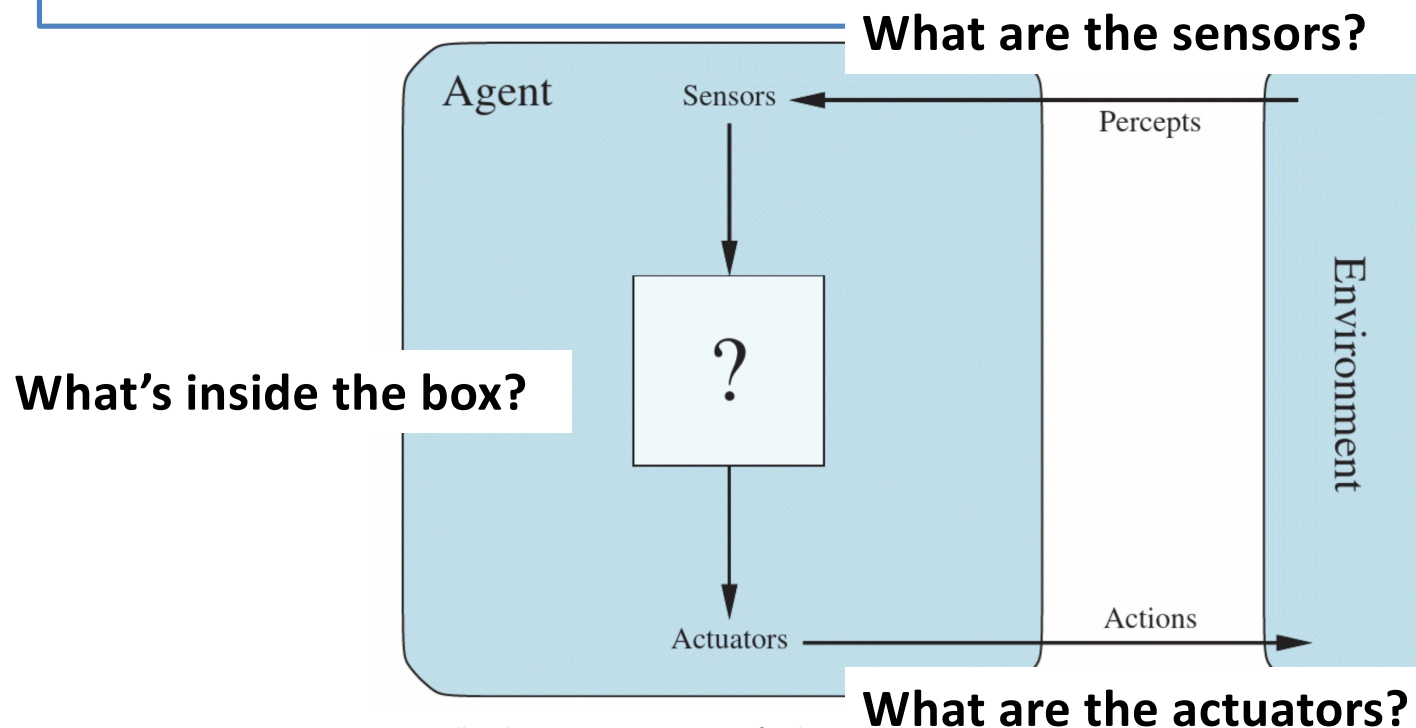
“An AI agent is a software component that has the agency to act on behalf of a user or a system to perform tasks. Users can organize agents into systems that can orchestrate complex workflows, coordinate activities among multiple agents, apply logic to thorny problems, and evaluate answers to user queries.” – McKinsey & Company

“Artificial intelligence (AI) agents are small, specialised pieces of software that can make decisions and operate cooperatively or independently to achieve system objectives. Agentic AI refers to AI systems composed of agents that can behave and interact autonomously in order to achieve their objectives.” – Gov.UK

And so many more...

Did we need a new definition?

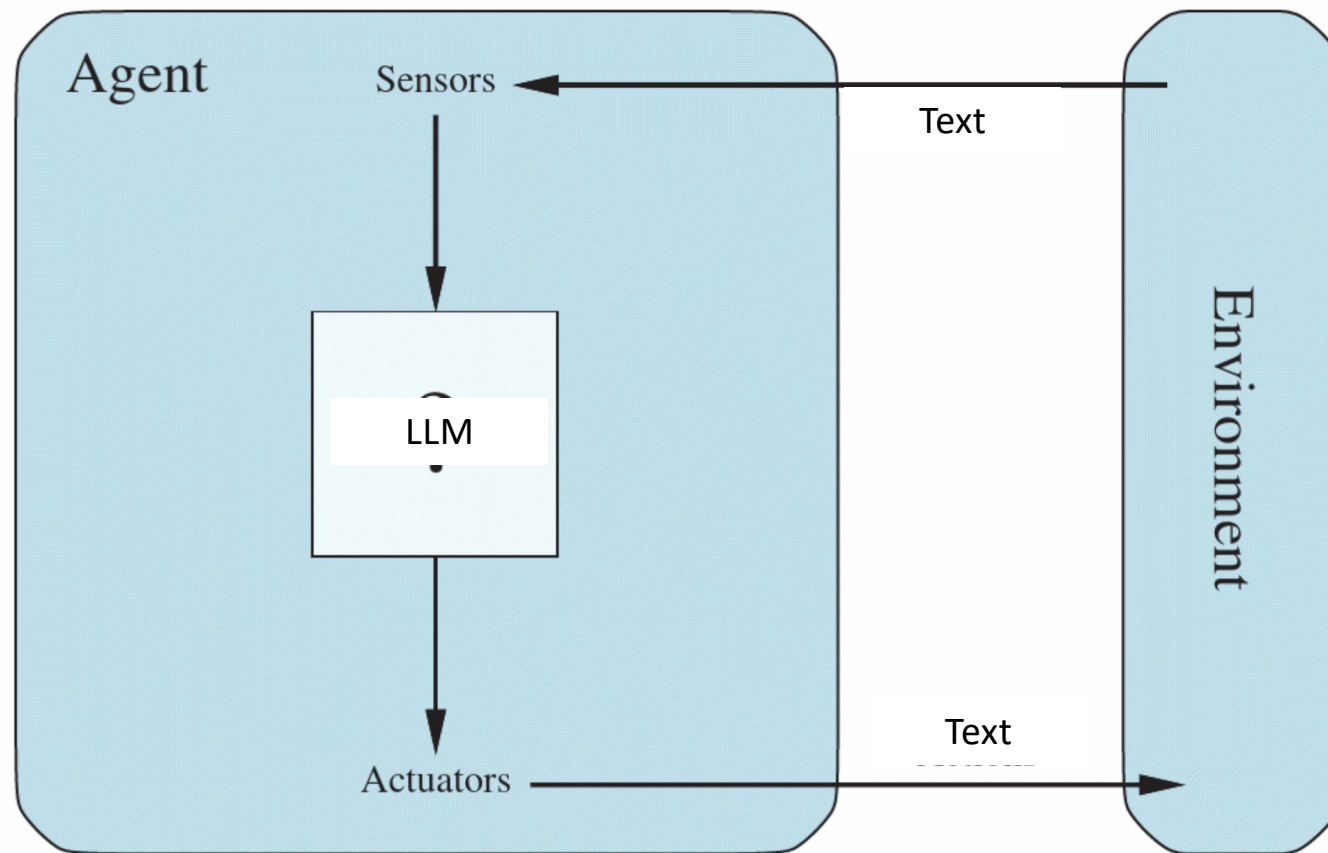
“An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.”
-- Russell and Norvig



Stuart Russell and Peter Norvig. 2024. Artificial Intelligence

USA.

The Anatomy of an LLM-based AI Agent



Main advantages: 1) leverage pretrained knowledge, 2) uses text to represent inputs/outputs

Agentic AI Problem-Solving Process

- Get the Mission: Use natural language to specify goals and constraints
- Scan the Scene: The agent can autonomously determine the relevant sources of information required (e.g., RAG)
- Think it Through: Use reasoning steps to determine the best plan given the information available
- Take Action: Generate an action that will bring the agent closer to the goal
- Learn & Get Better: Learn from the interaction with the environment to solve tasks better (ideally!)

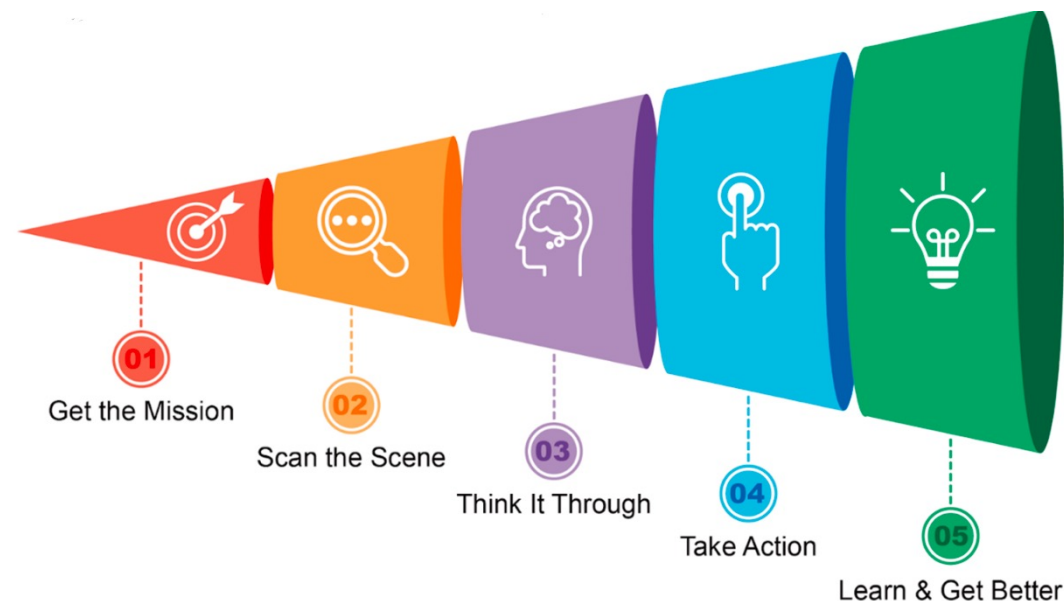


Fig. 1 Agentic AI functions as an intelligent assistant, continuously learning through experience. It operates via a straightforward five-step loop to accomplish tasks

Gulli, 2025. "Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems", Springer.

Agentic AI Problem-Solving Process

- Get the Mission: Use natural language to specify
- Scan the autonom sources o (RAG)
- Think it T to determ informati
- Take Acti will bring
- Learn & Get Better. Learn from the interaction with the environment to solve tasks better (ideally!)

This topic requires all the knowledge that we explored in this course:
LLMs, Prompting, RAG, long-context modelling, and multimodal!

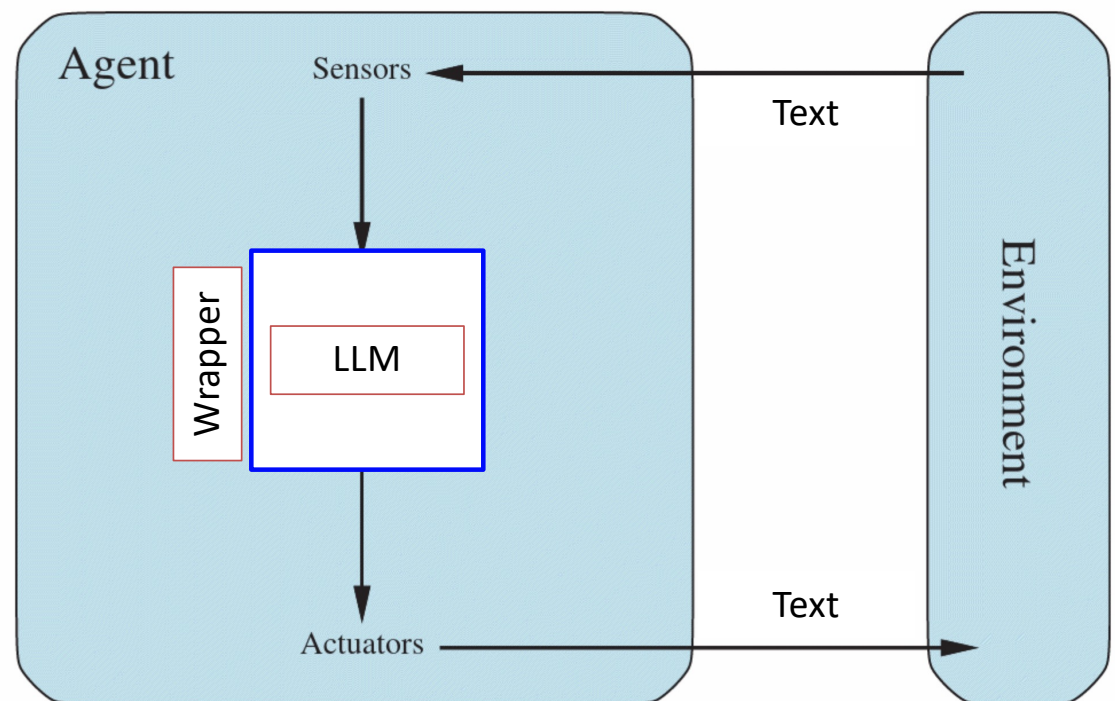


Fig. 1 Agentic AI functions as an intelligent assistant, continuously learning through experience. It operates via a straightforward five-step loop to accomplish tasks

Gulli, 2025. "Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems", Springer.

Pitfalls in Designing AI Agents

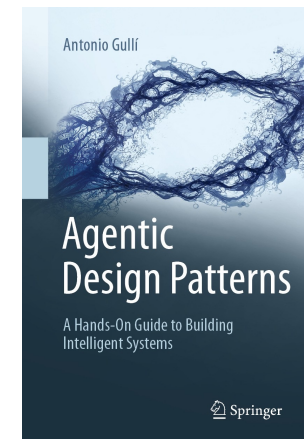
- Framework != LLM: you never interact with the LLM but with a wrapper around it!
- Modality gap: any input source of data needs to be converted into text to make sure that can be used by the LLM
- Context length limit: LLMs are inherently limited by the number of tokens that they can process
- Lack of memory: LLMs do not have notions of long-term memory (unless *simulated* by RAG)
- Action space: many agentic frameworks assume a specific action space that your LLM can select actions from



Reference material from Gulli, 2025. “Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems”, Springer. Available in the UoE Library.

DESIGN PATTERNS FOR AGENTIC AI SYSTEMS

Battle-tested blueprints for production-ready Agentic AI Systems



Why Patterns Matter in Agent Development

“An agentic system is a computational entity designed to perceive its environment (both digital and potentially physical), make informed decisions based on those perceptions and a set of predefined or learned goals, and execute actions to achieve those goals autonomously”

- Unlike traditional software, which follows rigid, step-by-step instructions, agents exhibit a degree of flexibility and initiative
- This flexibility in the behaviour of the model introduces complexities in the development which requires battle-tested blueprints
- By applying these blueprints you will make sure to enhance the structure, maintainability, reliability and efficiency of the agents you will build

Single-call execution

- Given a task T , we can create a natural language prompt for our task p_T
- We select our LLM based on our compute budget and task characteristics
 - ELM provides good options for us!
- And we treat it as a function where $\text{LLM}(p_T)$ will produce an output o_T
- This works well for simple, one step tasks and it can be easily tested
- Does this scale for complex problems requiring multiple steps?

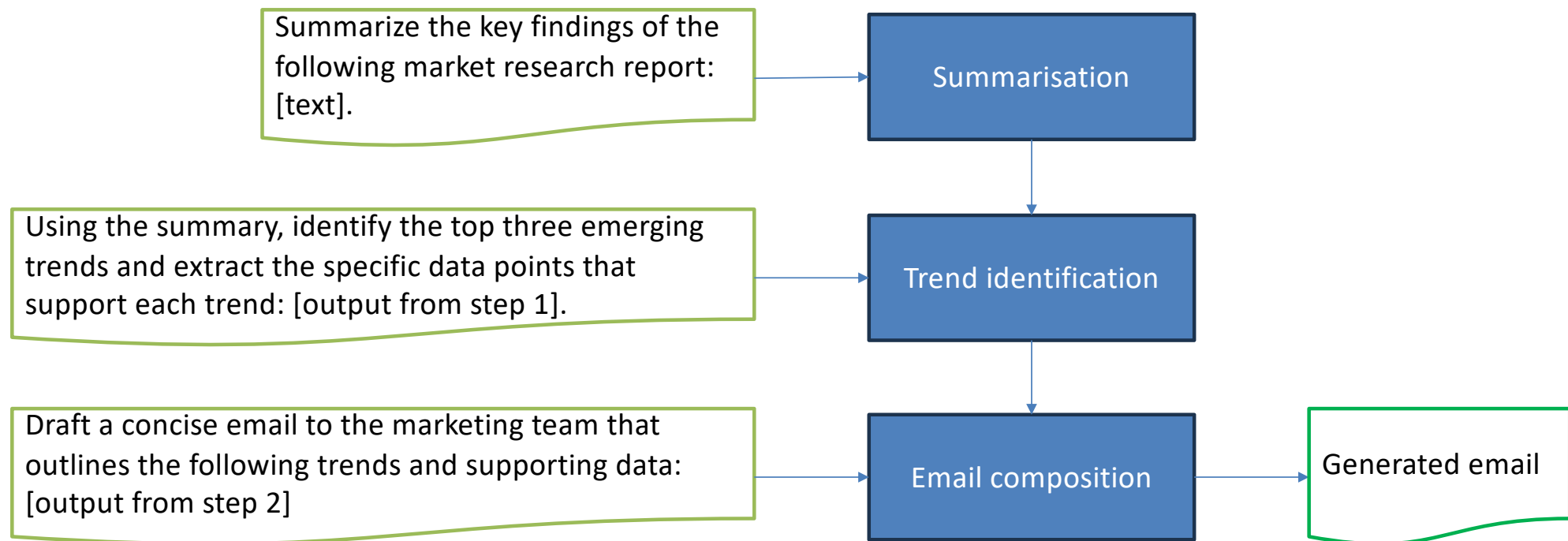


Pattern 1: Prompt Chaining

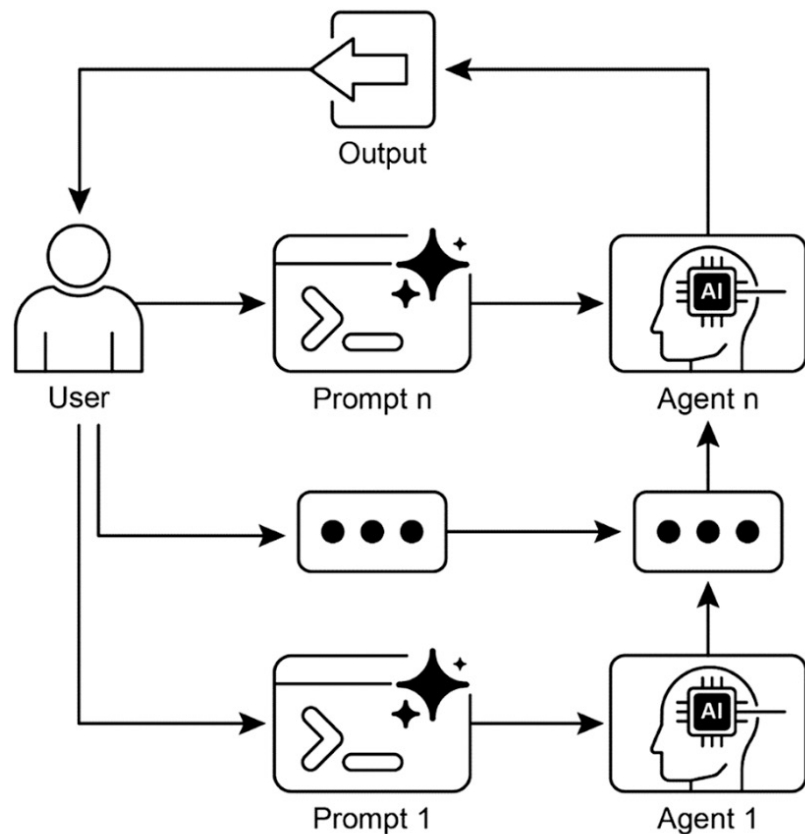
- **Prompt chaining** or also called **Pipeline** pattern represents a powerful design pattern for handling complex tasks
- The core idea is to break down the original problem into a sequence of smaller and more manageable sub-problems (*divide-et-impera*)
- We can create a sub-module with a custom prompt for each sub-problem
- And then, the output generated from one prompt is strategically fed as input into the next module in the chain

Prompt Chaining – A Practical Example

Task: Analyse the recent market research report by McKinsey on GenAI, summarize findings, identify trends with data points, and draft an email to our marketing team



Prompt Chaining – Strengths and Weaknesses



1. Modularity
2. Clarity
3. Ease of debugging
4. Reliability

1. Inefficiency
2. Inability to keep track of context
3. Error propagation
4. Requires specific constraints on inputs/outputs between modules

Pattern 2: Routing

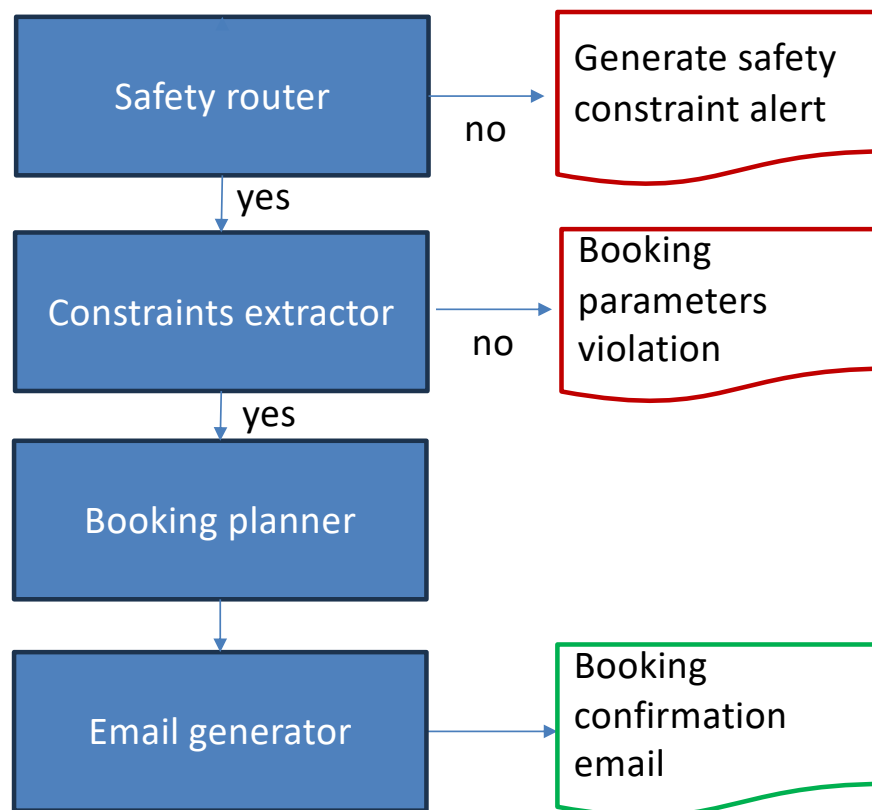
- The **Routing** pattern allows developers to go beyond the sequential nature of the prompt chaining
- Routing introduces conditional logic which allows more flexible and context-aware system behaviour
- Conditional logic enables developers to introduce safety guardrails that can prevent your agent to execute unsafe behaviours
- How do we define a conditional check in our workflow?

Routing Implementations

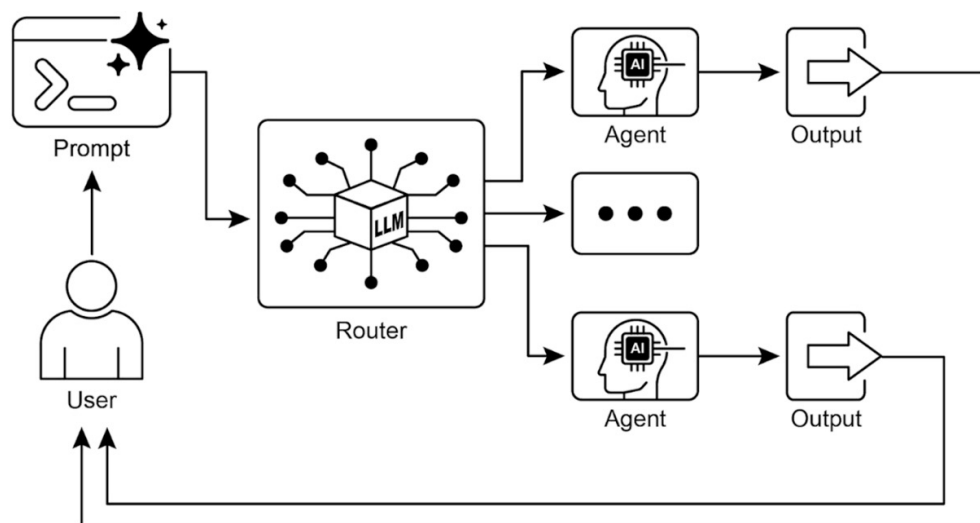
- Routing introduces conditional logic which allows more flexible and context-aware system behaviour
- Routing determines the **user intent** and triggers the correct **behaviour**
- Routing can be implemented in several ways:
 1. Rule-based routing: uses predefined rules based on keywords, patterns or structured data extracted from the input
 2. ML Model-based routing: it employs a discriminative model (e.g., classifier) trained to classify the request of the user (intent classification)
 3. Embeddings-based routing: The input query can be converted into a vector (as in RAG) and compared to embeddings representing different routes or capabilities
 4. LLM-based routing: The language model itself can be prompted to analyse the input and output a specific intent label

Routing – A Practical Example with Flight Booking

Task: Book an holiday for two people for a 2 weeks holiday in the Caribbeans. Make sure that you select only direct connections via London. Then generate an email to confirm my booking details.



Routing – Strengths and Weaknesses



1. Makes the system more dynamic and reactive
2. Allows to handles a variety of potential inputs and direct them to the most performant component
3. Flexible ways to implement the routing strategy

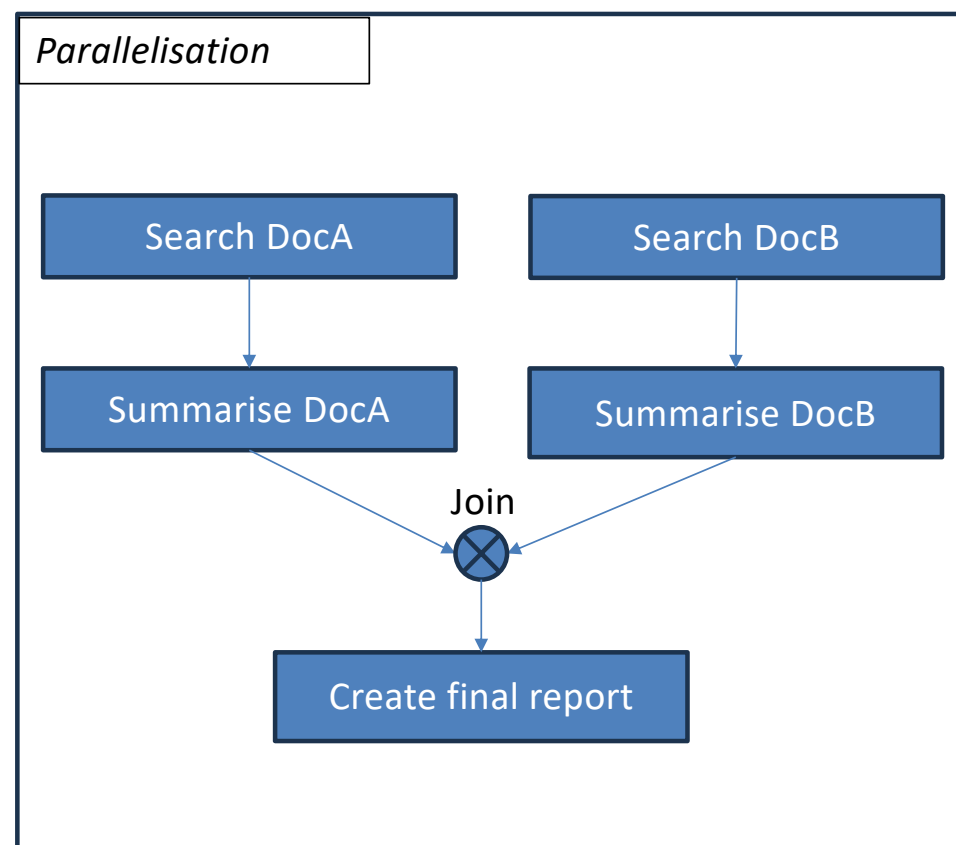
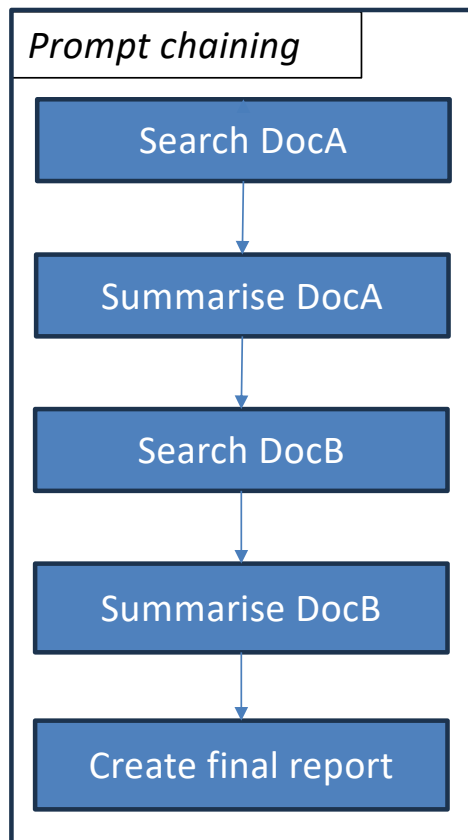
1. Trade-offs between different routing implementations
2. Introduces a potential point of failure for misclassifications

Pattern 3: Parallelisation

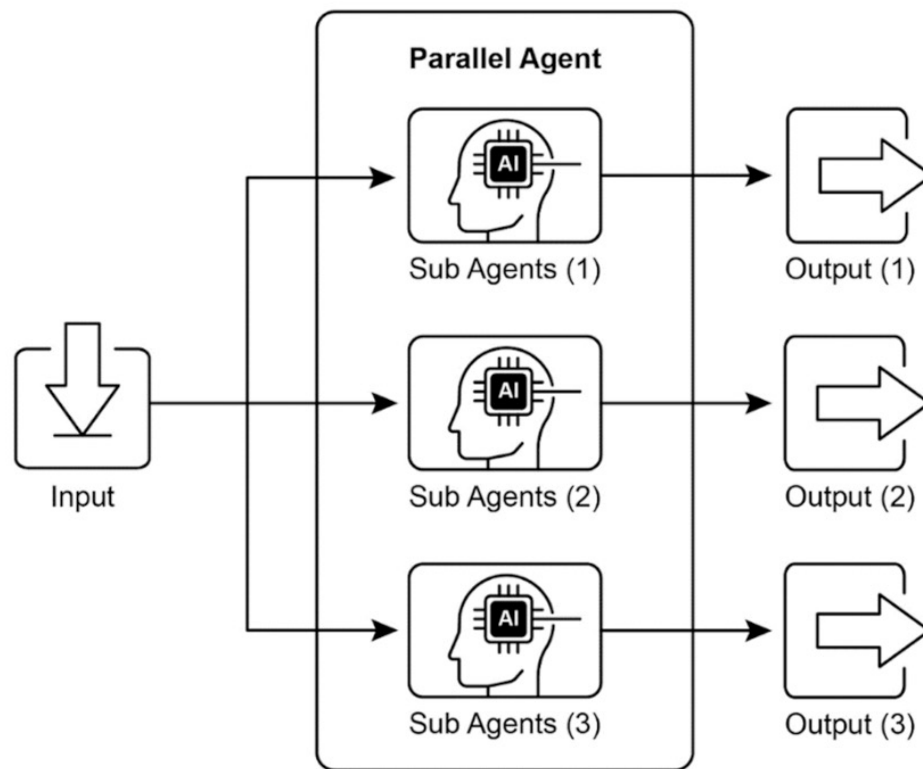
- Many complex agentic tasks can be executed *simultaneously* rather than one after the other (like in the chaining pattern)
- **Parallel execution** allows independent tasks to run at the same time, significantly reducing the execution time if your problem can be decomposed in many independent sub-problems
- Each sub-task can be arbitrarily complex (e.g., even *entire AI agents*)
- Parallel execution requires substantial overhead in combining the results returned by the different sub-modules

Parallelisation – Multi-document Summarisation

Task: Search DocA and DocB, then generate a summary that includes the main facts from both documents as well as important differences.



Parallelisation – Strengths and Weaknesses



1. Speeds-up processing of independent tasks
2. Avoid sequential calls

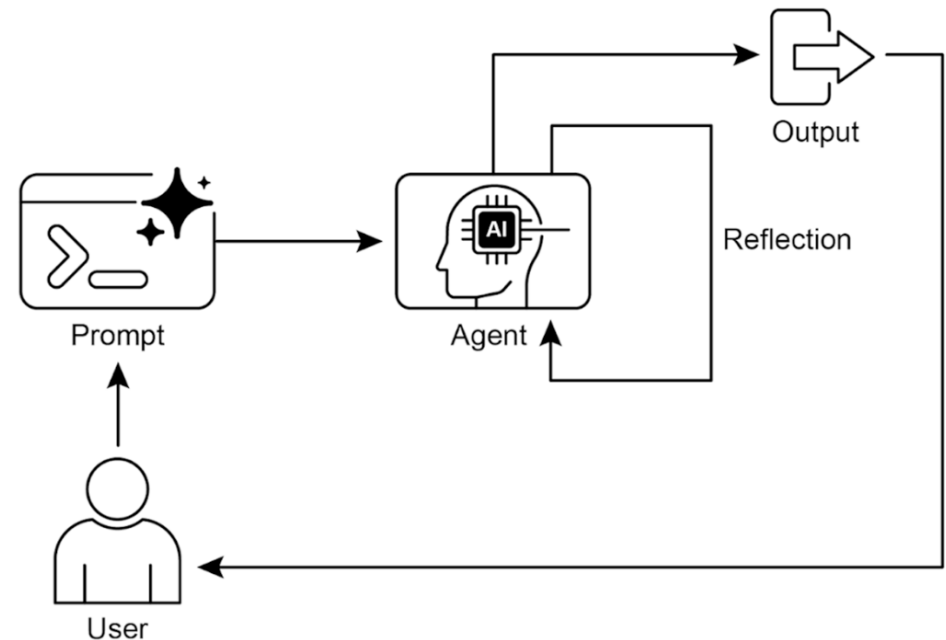
1. Greater computational load at time of fork
2. Introduces overhead when joining results
3. Joining results might create issues when agents are stuck

Pattern 4: Reflection

- So far we've assumed that our agents are *perfect*: given an input task they always generate a response
- However, this ignores that our agents will be challenged with unpredictable scenarios such as:
 - Unresponsive databases
 - Unresponsive source websites
 - Not up to date APIs
 - ... other?
- Reflection is a pattern that involves an agent evaluating its own work, output, or internal state and using that evaluation to improve its performance or refine its response
- It becomes a form of self-improvement based on some form of *feedback* that the agent can use to improve its behaviour via trial-and-error

Reflection using Environment Feedback

- The user provides an input prompt to the agent
- The agent generates an output which is executed in the environment
- The environment generates some feedback (e.g., execution errors) that is used by the agent to refine its own output
- Optionally, the agent attempts the task multiple times before returning control to the user



ReAct Prompting Technique

(1) Hotspot QA

Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

(1a) Standard

Answer: iPod



(1b) CoT (Reason Only)

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



(1c) Act-Only

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control ...

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ...

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Act 4: Finish[yes]



(1d) ReAct (Reason + Act)

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

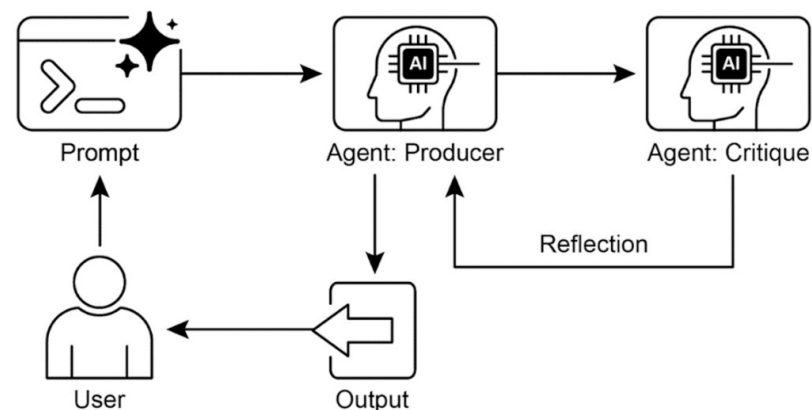
Act 4: Finish[keyboard function keys]



Yao, Shunyu, et al. "ReAct: Synergizing reasoning and acting in language models." The eleventh international conference on learning representations. 2022.

Reflection via Producer-Critique

- The workflow is similar to the previous implementation but now we have two distinct agents
- The **Producer agent** is responsible for generating the output for the user prompt
- The **Critique agent** is responsible for evaluating the output generated by the Producer according to specific criteria (e.g., factual accuracy)
- The feedback from the Critique is then passed back to the Producer which generates a refined version
- Optionally, multiple feedback loop can be implemented



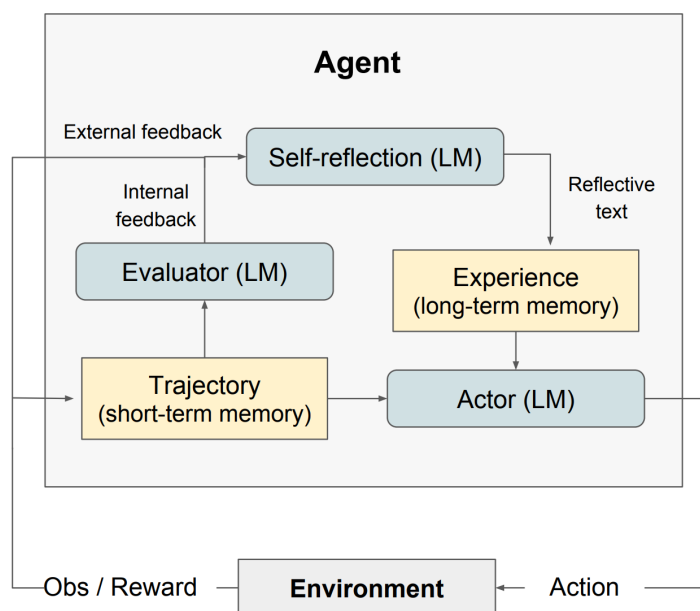
Reflexion: Language Agents with Verbal Reinforcement Learning

- The author of this paper study a very interesting question:
 - considering that LLM are very good at interpreting natural language, would they be able to use natural language feedback to change their behaviour?
- Concretely, this can be considered one of the most important problem in AI: the **credit assignment problem**
- They propose a framework that updates the model behaviour by simply injecting linguistic feedback as part of the prompt without any parameter updates
 - In this way they can also convert numerical rewards generated by the environment (ala RLVR!) into linguistic feedback that the LLM can process directly

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." Advances in Neural Information Processing Systems 36 (2023): 8634-8652.

Reflexion: Language Agents with Verbal Reinforcement Learning

Notation: M_a = the actor that generates text and actions, M_e = scores the outputs produced by the actor, M_{sr} = the self-reflection model that generates natural language feedback



Algorithm 1 Reinforcement via self-reflection

```
Initialize Actor, Evaluator, Self-Reflection:  
 $M_a, M_e, M_{sr}$   
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$   
Generate initial trajectory using  $\pi_\theta$   
Evaluate  $\tau_0$  using  $M_e$   
Generate initial self-reflection  $sr_0$  using  $M_{sr}$   
Set  $mem \leftarrow [sr_0]$   
Set  $t = 0$   
while  $M_e$  not pass or  $t < \text{max trials}$  do  
    Generate  $\tau_t = [a_0, o_0, \dots a_i, o_i]$  using  $\pi_\theta$   
    Evaluate  $\tau_t$  using  $M_e$   
    Generate self-reflection  $sr_t$  using  $M_{sr}$   
    Append  $sr_t$  to  $mem$   
    Increment  $t$   
end while  
return
```

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." Advances in Neural Information Processing Systems 36 (2023): 8634-8652.

Reflexion: Language Agents with Verbal Reinforcement Learning

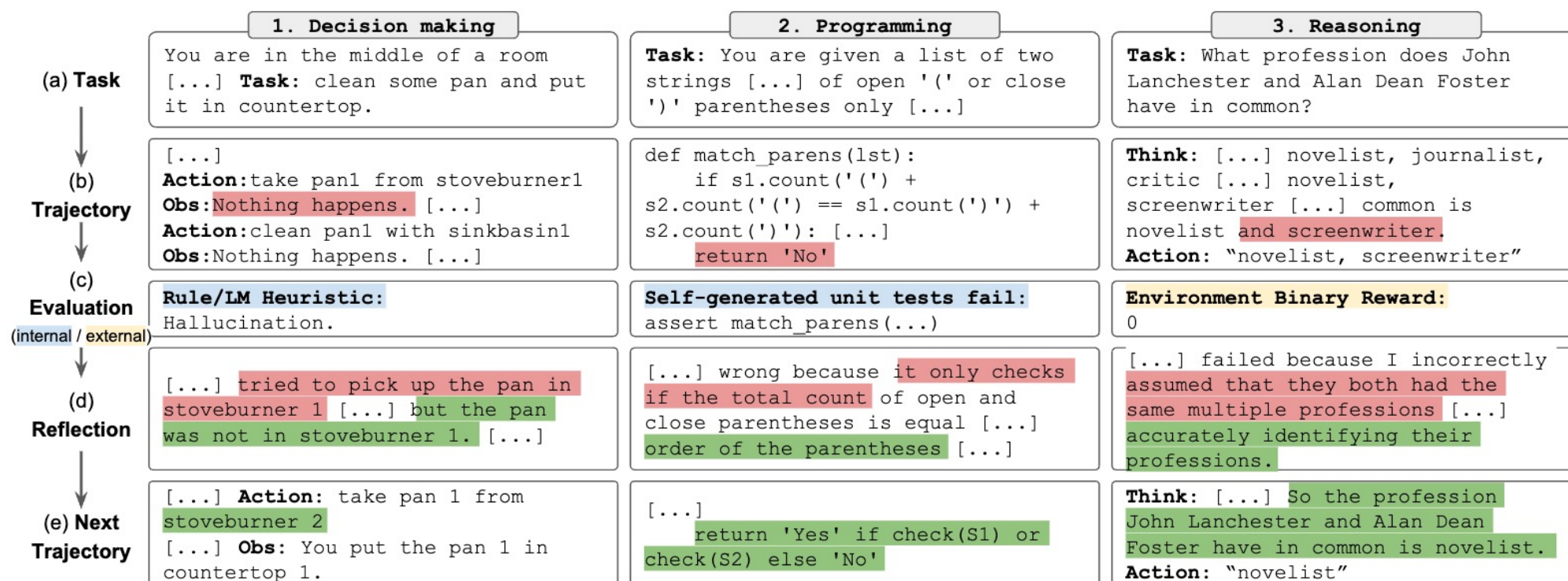


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

Shinn, Noah, et al. "Reflexion: Language agents with verbal reinforcement learning." Advances in Neural Information Processing Systems 36 (2023): 8634-8652.

Pattern 5: Tool Use Pattern

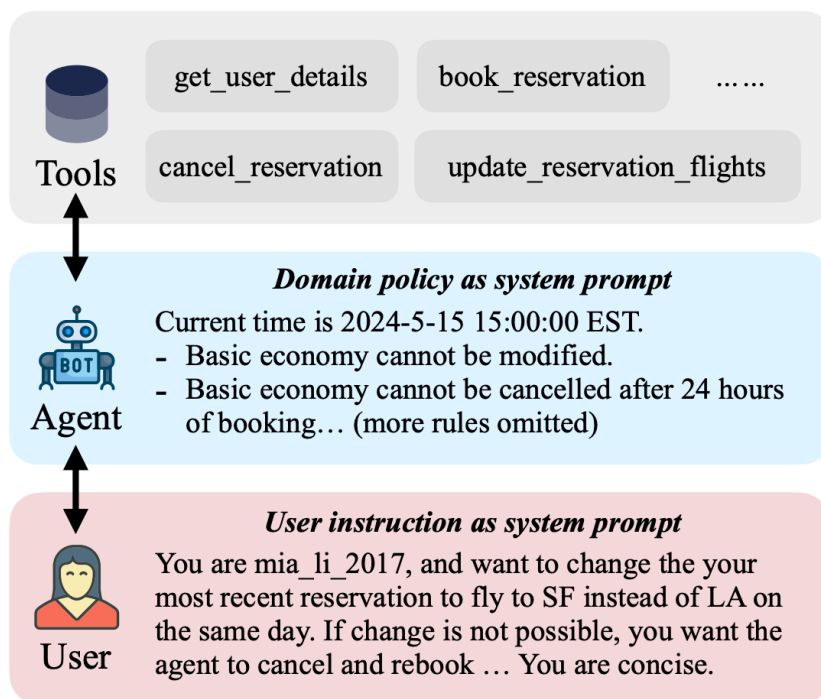
- All the previous patterns mostly concerned with orchestrating the interaction between the LLMs, managing the flow of information within the agent's internal workflow and the interaction with the environment
- However, real-world applications require more access to existing services such as external databases or external APIs
- The Tool Use Pattern allows an LLM to decide when and how to use a specific external function based on the current state (e.g., the user's request intent)
- This pattern is easy to implement because it relies on the ability of LLMs to generate **structured output** (typically code in JSON or Python)

Tool Use Pattern – how does it work?

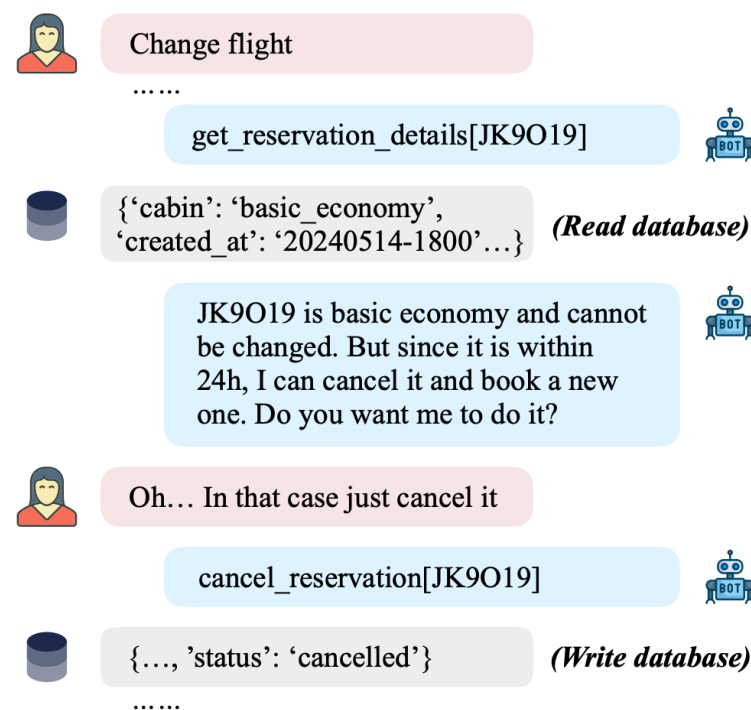
- The Tool Use Pattern allows an LLM to decide when and how to use a specific external function based on the current state (e.g., the user's request intent)
- This pattern is easy to implement because it relies on the ability of LLMs to generate code (typically in Python):
 - Tool definition: external functions and services are defined and described to the LLM
 - LLM decision: the LLM receives the tool definitions as well as the user request as input, and decides which tools should be invoked
 - Function Call Generation: if the LLM decides to use a tool, it generates a structured output (e.g., JSON object or Python) that specifies the name of the tool to call and the arguments (i.e., parameters) to pass to it
 - Tool execution: The agentic framework retrieves the referenced tool and executes it
 - Observation/Result processing: We reformat the output for the LLM
 - LLM processing (Optional): The LLM receives the tool output as context and uses it to generate the final answer -> this fits very well with another pattern; which one?

τ -Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains

(a) τ -bench setup

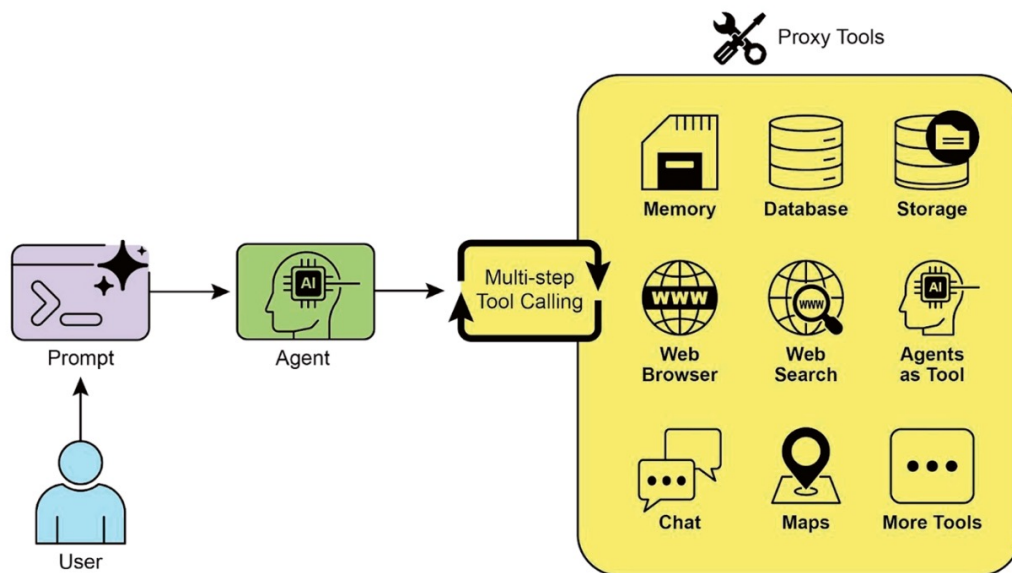


(b) Example trajectory in τ -airline



Yao, Shunyu, et al. " τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." arXiv preprint arXiv:2406.12045 (2024).

Tool Use – Strengths and Weaknesses



1. Allows the LLM to access always up-to-date
2. Extends the LLM functionalities
3. Integrates well with other agentic design patterns

1. Increases the number of tokens that the LLM has to process at inference time
2. Requires that the LLM is able to process and generate code that can be executed

Model Context Protocol (MCP)

- Using language to represent inputs and actions is an effective way to leverage the existing knowledge contained in LLMs
- However, to interact with real-world systems (e.g., databases), LLMs need to use very specific commands and interfaces
- MCP != Tool use: MCP represents an open standard designed to standardise how LLMs can communicate with external applications, data sources, and tools
- MCP can be intended as a general-purpose communication protocol that allows LLMs to obtain context, execute actions, and interact with various systems
- MCP is now considered an open standard that can be found here:
<https://github.com/modelcontextprotocol/modelcontextprotocol>

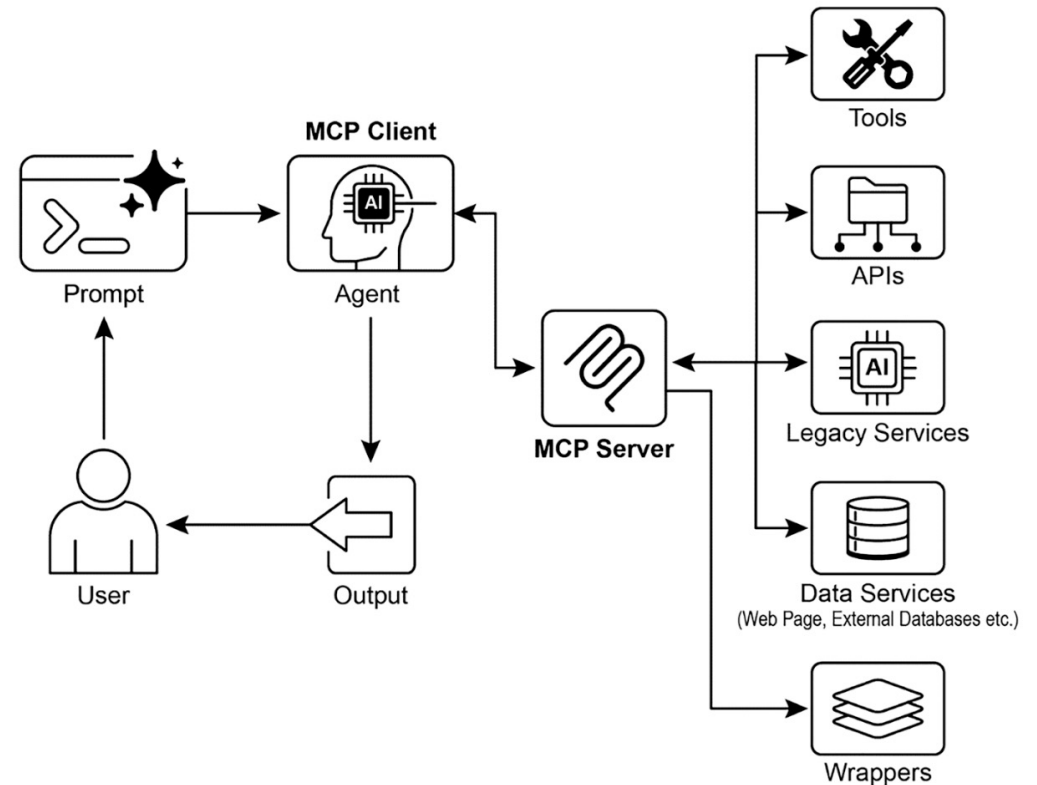
MCP vs Tool use

Feature	Tool function calling	Model context protocol (MCP)
Standardization	Proprietary and vendor-specific. The format and implementation differ across LLM providers.	An open, standardized protocol, promoting interoperability between different LLMs and tools.
Scope	A direct mechanism for an LLM to request the execution of a specific, predefined function.	A broader framework for how LLMs and external tools discover and communicate with each other.
Architecture	A one-to-one interaction between the LLM and the application's tool-handling logic.	A client-server architecture where LLM-powered applications (clients) can connect to and utilize various MCP servers (tools).
Discovery	The LLM is explicitly told which tools are available within the context of a specific conversation.	Enables dynamic discovery of available tools. An MCP client can query a server to see what capabilities it offers.
Reusability	Tool integrations are often tightly coupled with the specific application and LLM being used.	Promotes the development of reusable, standalone "MCP servers" that can be accessed by any compliant application.

Image from Gulli, 2025. "Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems", Springer. Available in the UoE Library.

MCP's Importance for Real-world Applications

- Use MCP when building complex, scalable, and enterprise-grade agentic systems
- Essential when these agentic systems need to operate within a broader ecosystem consisting of **heterogeneous and distributed services**
- The Model Context Protocol **uses a client-server model** to standardize information flow to orchestrate intelligent behaviour
- The standard facilitates the definition of useful components such as:
 1. Prompts
 2. Resources
 3. Tools



Pydantic AI: GenAI Agentic Framework

- Many agentic AI frameworks have been proposed to date. **PydanticAI** is a recent framework that facilitates the implementation of agentic LLM solutions with a very clear API and a focus on data validation
- For today's lecture, I have prepared a codebase to showcase some of the design patterns that we've seen today:
 - <https://github.com/aleSuglia/uae-agents>
- This example uses an **LLM deployed locally** using **Ollama**. However, you can imagine to edit this code to support other LLMs (e.g., ELM?) by following the official documentation from Pydantic AI:
 - <https://ai.pydantic.dev/>

Pydantic AI: GenAI Agentic Framework

- Many agentic recent frameworks provide solutions with
- For today's leading design patterns
 - <https://github.com/pydantic/pydantic-ai>
- This example is designed to help you imagine to explore the official documentation
 - <https://ai.pydantic.dev>



PydanticAI is a
GenAI LLM

Some of the

However, you can
try following the

Summary

- Agentic AI is becoming an important topic of interest for researchers and practitioners. A lot of hype around it which we need to carefully navigate.
 - Different frameworks have been developed
 - Different definitions have been proposed
 - Ultimately, they all boil down to Russel&Norvig definition!
- When defining an agentic AI application, it's important to consider the information flow and when specialised decision making is required
- For this reason, many different “design patterns” are possible and they should be used to maximise efficiency and efficacy
- Still a lot to be done in this space to ensure reliability and safety of agentic AI applications
- Next lecture: can we go beyond agents that only process text?

Homework

- Reference material for these slides is from Gulli, 2025. “Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems”. This book is freely available in the library:
https://discovered.ed.ac.uk/permalink/44UOE_INST/1lk19of/alma9925627813602466
 - Most of the slides are from Part I of this book; focus on the topics covered in the slides!
- The agent definition is from “Russell, Stuart J, et al. Artificial Intelligence : A Modern Approach.”:
https://discovered.ed.ac.uk/permalink/44UOE_INST/1620n3/alma9923946712902466
 - Chapter 2: “Intelligent Agents”

Additional references

- The reference textbook from Gulli et al. contains interesting code snippets that you can use to learn about the different agentic design patterns. Their code examples are written using Google's AI framework ADK: <https://google.github.io/adk-docs/>
- In this lecture, we have used instead Pydantic AI but many other frameworks exist in the literature:
 - LangChain: <https://www.langchain.com/>
 - OpenAI-Agents: <https://github.com/openai/openai-agents-python>
 - Microsoft Autogen: <https://github.com/microsoft/autogen>
 - And many more...
- Remember: always consider the trade-off between number of features available and ease of use when adopting one of these frameworks!